Universidad Nacional Abierta y a Distancia - UNAD

Escuela de Ciencias Basicas, Tecnologia e Ingenieria (ECBTI)

Programa: Ingenieria de Sistemas

Curso: Programacion

Codigo: 213023

Fase 4 - Componente practico: Practicas simuladas

Sistema Integral de Gestion de Clientes, Servicios y Reservas - Software FJ

Estudiante: SARLYN LEIDYS MURILLO PALACIOS

Tutor: Wilson Hernan Perez Correa

Fecha de entrega: 10 de noviembre de 2026

Introduccion

La programacion orientada a objetos (POO) constituye un paradigma fundamental en el desarrollo de software, ya que facilita la organizacion del codigo mediante abstracciones que representan entidades del mundo real. A traves de conceptos como la encapsulacion, la herencia y el polimorfismo, es posible construir sistemas modulares, reutilizables y mas sencillos de mantener, favoreciendo la calidad y la escalabilidad de las soluciones tecnologicas.

De igual manera, el manejo avanzado de excepciones es esencial para garantizar la estabilidad y la confiabilidad de las aplicaciones. La anticipacion, control y registro de errores permiten que un sistema continue su ejecucion ante situaciones imprevistas, protegiendo la integridad de los datos y mejorando la experiencia de uso. En este contexto, la implementacion de logs proporciona trazabilidad sobre las operaciones realizadas y los fallos detectados.

El presente trabajo desarrolla el Sistema Integral de Gestion de Clientes, Servicios y Reservas para la empresa ficticia Software FJ, como parte de la Fase 4 - Componente practico: Practicas simuladas del curso Programacion (213023) de la UNAD. El sistema aplica rigurosamente los principios de la POO y el manejo avanzado de excepciones en Python, administrando la informacion operativa en memoria y utilizando archivos unicamente para el registro de eventos y errores.

Objetivos

Objetivo general

Desarrollar un Sistema Integral de Gestion de Clientes, Servicios y Reservas para la empresa ficticia Software FJ, aplicando los principios de programacion orientada a objetos y el manejo avanzado de excepciones en Python, garantizando un funcionamiento estable, modular y documentado.

Objetivos especificos

- Disenar e implementar una arquitectura modular en Python que integre clases, herencia, encapsulacion y polimorfismo para la gestion de clientes, servicios y reservas.

- Aplicar mecanismos de validacion y manejo avanzado de excepciones personalizadas, incluyendo estructuras try/except/else/finally, raise y encadenamiento de excepciones.

- Verificar el correcto funcionamiento del sistema mediante la simulacion de operaciones, la ejecucion de pruebas automatizadas y el registro de eventos en un archivo de logs.

Desarrollo

Arquitectura del sistema

El proyecto se estructuro de forma modular para favorecer la separacion de responsabilidades, la legibilidad y el mantenimiento. El paquete principal 'software_fj' contiene los modulos de entidades, clientes, servicios, reservas, excepciones, gestor y configuracion de logs. La informacion operativa se gestiona en memoria mediante objetos y estructuras de datos, mientras que el archivo 'logs/software_fj.log' se utiliza exclusivamente para la trazabilidad.

Clases, herencia, encapsulacion y polimorfismo

Se implemento la clase abstracta general 'Entidad', que define el contrato comun de las entidades del sistema. La clase 'Cliente' hereda de 'Entidad' y encapsula sus datos personales mediante atributos protegidos y propiedades con validaciones estrictas. Para los servicios, se definio la clase abstracta 'Servicio' y tres clases derivadas: 'ReservaSala', 'AlquilerEquipo' y 'AsesoriaEspecializada'.

El polimorfismo se evidencia en el calculo de costos, donde cada tipo de servicio sobrescribe el metodo 'costo_base' y utiliza reglas especificas. Asimismo, se sobrescriben metodos como 'descripcion' y 'validar'. Se emplearon parametros opcionales en metodos como 'calcular_costo', 'procesar' y 'actualizar_contacto', como forma de sobrecarga adaptada al lenguaje Python.

Excepciones personalizadas y manejo avanzado

Se definieron las excepciones 'DatoInvalidoError', 'ClienteNoEncontradoError', 'ServicioNoDisponibleError', 'ReservaNoPermitidaError' y 'ParametroFaltanteError'. El sistema utiliza estructuras 'try/except', 'try/except/else' y 'try/except/finally', ademas de la instruccion 'raise' para senalar condiciones invalidas. Tambien se implemento encadenamiento de excepciones mediante 'raise ... from ...' para preservar la causa original de los errores y mejorar el diagnostico.

Gestion de logs

El modulo 'logger_config.py' configura el registro de eventos utilizando la biblioteca estandar 'logging'. Los mensajes se almacenan en 'logs/software_fj.log' incluyendo fecha y hora, nivel, nombre del logger y descripcion del evento. Se registran operaciones exitosas, validaciones, cambios de estado de reservas y excepciones controladas.

Pruebas realizadas y diez operaciones simuladas

Se desarrollaron pruebas automatizadas con 'unittest' para validar el comportamiento del sistema (registro y validaciones de clientes, creacion de servicios, calculo de costos, estados de reservas y generacion de logs). Adicionalmente, el archivo 'main.py' ejecuta las diez operaciones solicitadas en la guia:

- Registro de un cliente valido.

- Intento de registro de un cliente con correo invalido (controlado).
- Registro de otro cliente valido.
- Creacion de un servicio de reserva de salas.
- Creacion de un servicio de alquiler de equipos.
- Creacion de un servicio de asesorias.
- Intento de creacion de un servicio con tarifa incorrecta (controlado).
- Creacion, confirmacion y procesamiento de una reserva correctamente.
- Intento de creacion de una reserva con un servicio inexistente (controlado).
- Intento de procesamiento de una reserva sin confirmarla previamente (controlado).

Las ejecuciones reales realizadas en el entorno de desarrollo evidenciaron que los errores controlados no interrumpen el flujo del programa y que las pruebas automatizadas se completaron satisfactoriamente (10 pruebas, estado OK).

Evidencias

A continuacion se presentan los espacios destinados para incluir capturas autenticas de la ejecucion y verificacion del sistema. No se incluyen capturas inventadas; los espacios deben completarse con evidencias reales obtenidas por la estudiante.

[Espacio para captura autentica: Ejecucion del programa (python main.py)]

[Espacio para captura autentica: Operaciones exitosas en consola]

[Espacio para captura autentica: Operaciones fallidas controladas en consola]

[Espacio para captura autentica: Contenido del archivo de logs (logs/software_fj.log)]

[Espacio para captura autentica: Ejecucion de pruebas automatizadas (unittest)]

[Espacio para captura autentica: Repositorio GitHub (captura del repositorio publicado)]

Enlace de GitHub

Campo para el enlace real del repositorio (completar manualmente tras la publicacion):

[PEGAR AQUI EL ENLACE REAL DEL REPOSITORIO GITHUB]

Nota: En este entorno no se realizo la publicacion automatica del repositorio ni se genero un enlace. El proyecto se entrega preparado para su carga en GitHub siguiendo las instrucciones incluidas en el README.md.

Conclusiones

- La aplicacion de los principios de programacion orientada a objetos permitio estructurar el sistema Software FJ de manera modular y coherente, facilitando la representacion de clientes, servicios y reservas mediante abstracciones claras y reutilizables.

- El uso de validaciones estrictas y del manejo avanzado de excepciones, junto con el encadenamiento de errores y el registro en logs, contribuyo a la estabilidad del sistema, permitiendo controlar fallos sin interrumpir la ejecucion de las operaciones.

- La verificacion mediante la simulacion de las diez operaciones y la ejecucion de pruebas automatizadas confirmo el cumplimiento de los requisitos funcionales y tecnicos del proyecto, evidenciando el correcto calculo de costos, la gestion de estados y la trazabilidad de eventos.

Referencias bibliograficas

Cuevas Alvarez, A. (2016). Python 3: curso practico (pp. 373-385). RA-MA Editorial. https://elibro-net.bibliotecavirtual.unad.edu.co/es/ereader/unad/106404

Romano, F., Baka, B., & Phillips, D. (2019). Getting started with Python: Understand key data structures and use Python in object-oriented programming. Packt Publishing.

Silva, F. D. (2025). Basics of using GitHub [Objeto Virtual de Informacion]. Repositorio Institucional UNAD. https://repository.unad.edu.co/handle/10596/75882

Van Rossum, G., & Drake Jr., F. L. (2024). El tutorial de Python. Python Software Foundation. https://docs.python.org/es/3.12/tutorial/errors.html

Zambrano, J. P. (2025). Introduccion al uso de GitHub [Objeto Virtual de Informacion]. Repositorio Institucional UNAD. https://repository.unad.edu.co/handle/10596/75876